

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Submitted By

Name: Aresh Khobragade

Internship:-

VLSI Design Internship

Task Number :- Task-2

Sr. No.	INDEX	
1.	Abstract	
2.	Introduction	
3.	Objectives	
4.	Software Tools Used	
5.	Theory	
6.	Logic Gates Implementation	
7.	Half Adder	
8.	Full Adder	
9.	Testbench	
10.	Stimulation Procedure	
11.	Results	
12.	Applications	
13.	Conclusion	
14.	References	
15.	Outcome:	

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Abstract

This project focuses on learning Verilog Hardware Description Language (HDL) and designing basic combinational circuits using Register Transfer Level (RTL) modeling. Several logic gates such as AND, OR, NOT, NAND, NOR, and XOR were implemented using Verilog. Basic arithmetic circuits including Half Adder and Full Adder were also designed and verified using testbenches. Functional simulation was performed using Xilinx Vivado to ensure correct operation.

Introduction

Verilog HDL is a hardware description language used for modeling and designing digital circuits. RTL (Register Transfer Level) design describes how data moves between registers and how logic operations are performed.

Combinational circuits produce outputs based solely on current inputs without requiring memory elements. They form the foundation of digital systems and VLSI design.

Objectives

- Understand the fundamentals of Verilog HDL.
- Learn RTL modeling concepts.
- Design basic logic gates using Verilog.
- Implement Half Adder and Full Adder circuits.
- Develop testbenches for verification.
- Perform simulation and waveform analysis.

Software Tools Used

Tool	Purpose
Xilinx Vivado	Design entry and simulation
Verilog HDL	Hardware description language
Testbench	Functional verification
Waveform Viewer	Output analysis

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Theory

4.1 Verilog HDL

Verilog is a Hardware Description Language used to model electronic systems. It enables the design and verification of digital circuits at different levels of abstraction.

4.2 RTL Design

RTL (Register Transfer Level) describes digital circuits in terms of registers and the transfer of data between them.

4.3 Combinational Circuits

Combinational circuits generate outputs based only on present inputs and do not contain memory elements.

Examples:

- Logic Gates
- Half Adder
- Full Adder

Logic Gates Implementation

AND Gate

Verilog Code

```
module and_gate(  
input a,  
input b,  
output y  
);  
  
assign y = a & b;  
  
endmodule
```

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

OR Gate

```
module or_gate(  
input a,  
input b,  
output y  
);  
  
assign y = a | b;  
  
endmodule
```

NOT Gate

```
module not_gate(  
input a,  
output y  
);  
  
assign y = ~a;  
  
endmodule
```

NAND Gate

```
module nand_gate(  
input a,  
input b,  
output y  
);  
  
assign y = ~(a & b);  
  
endmodule
```

NOR Gate

```
module nor_gate(  
input a,
```

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

```
input b,  
output y  
);  
  
assign y = ~(a | b);  
  
endmodule
```

XOR Gate

```
module xor_gate(  
input a,  
input b,  
output y  
);  
  
assign y = a ^ b;  
  
endmodule
```

Half Adder

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Verilog Code

```
module half_adder(  
input a,  
input b,  
output sum,
```

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

```
output carry
);

assign sum = a ^ b;
assign carry = a & b;

endmodule
```

Full Adder

Truth Table

A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Verilog Code

```
module full_adder(  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output carry  
);  
  
    assign sum = a ^ b ^ cin;  
    assign carry = (a & b) | (b & cin) | (a & cin);  
  
endmodule
```

Testbench

```
module tb_full_adder;  
  
    reg a,b,cin;  
    wire sum,carry;  
  
    full_adder uut(  
        .a(a),  
        .b(b),  
        .cin(cin),  
        .sum(sum),  
        .carry(carry)  
    );  
  
    initial begin  
        a=0; b=0; cin=0;  
        #10 a=0; b=0; cin=1;  
        #10 a=0; b=1; cin=0;  
        #10 a=0; b=1; cin=1;  
        #10 a=1; b=0; cin=0;  
        #10 a=1; b=0; cin=1;  
        #10 a=1; b=1; cin=0;  
        #10 a=1; b=1; cin=1;  
    end  
endmodule
```

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

```
#10;  
end  
  
endmodule
```

Stimulation Procedure

1. Create a new Vivado project.
2. Add Verilog source files.
3. Add the testbench file.
4. Set the testbench as the top module.
5. Run Behavioral Simulation.
6. Observe output waveforms.
7. Verify the functionality of each circuit.

Results

- Successfully designed basic logic gates.
- Implemented Half Adder and Full Adder circuits.
- Generated testbenches for functional verification.
- Simulated the designs using Vivado.
- Observed correct output waveforms for all input combinations.

Applications

- Digital arithmetic circuits.
- Processors and ALUs.
- Communication systems.
- Memory circuits.
- VLSI and FPGA designs.

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Conclusion

The project provided practical exposure to Verilog HDL and RTL design concepts. Basic combinational circuits including logic gates, Half Adder, and Full Adder were successfully implemented and verified through simulation. This task strengthened understanding of digital design methodologies and laid the foundation for advanced VLSI system design.

References

1. Samir Palnitkar, **Verilog HDL: A Guide to Digital Design and Synthesis**, Prentice Hall.
2. M. Morris Mano, **Digital Design**, Pearson Education.
3. Xilinx Vivado Design Suite User Guide.
4. IEEE Standard Verilog HDL Language Reference Manual.
5. NPTEL – Digital System Design Course.
6. EDA Playground Documentation.
7. Icarus Verilog User Guide.
8. GTKWave Documentation.

Outcome:

The objectives of Task-2 were achieved successfully by designing and verifying basic combinational circuits using Verilog HDL and RTL methodology.